CSE 214 HW01

1) By definition, $f(n) \in \mathcal{O}(g(n))$ if and only if

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} < +\infty.$$

Since $$\lim_{n\to\infty} \frac{n^2}{n^3} = \lim_{n\to\infty} \frac{1}{n} = 0 < +\infty,$$

$n^2 \in \mathcal{O}(n^3)$, while

$$\lim_{n\to\infty} \frac{n^3}{n^2} = \lim_{n\to\infty} n = +\infty$$

implies $n^3 \notin \mathcal{O}(n^2)$.

2a) This algorithm doesn't even stop in the worst case: since i is always equal to 1, if $A[1] \neq 1$, the while loop will run forever, so the algorithm is "$\mathcal{O}(\infty)$".

2b) This algorithm is $\boxed{\mathcal{O}(n)}$ because the while loop will run at most n times (the worst case being when $A[n]$ is the only element of A equalling target) and the contents of the loop, as well as the commands outside the loop, all run in $\mathcal{O}(1)$ time.

2c) There are n multiplications being done here, so the algorithm is $\mathcal{O}(n)$.

3i)
```
for i ← 1 to n do
    for j ← i+1 to n do
        if A[i] = A[j] then
            return A[i]
```

3ii)
```
sum ← 0
for i ← 1 to n do
    sum ← sum + A[i]
return sum − n(n−1)/2
```

Explanation: the returned number is $\underbrace{\sum_{i=1}^{n} A[i]}_{\text{sum}} - \underbrace{\sum_{i=1}^{n-1} i}_{\frac{n(n-1)}{2}}$.

If k is the repeated number, then $\sum_{i=1}^{n} A[i] = k + \sum_{i=1}^{n-1} i$, so the returned number is indeed k.

4) count ← 0
for i ← 1 to n do
    if ISPERFECTSQUARE(A[i]) then
        count ← count + 1
        temp ← A[count]
        A[count] ← A[i]
        A[i] ← temp

Explanation: count is the number such that the first count numbers of the array are all squares at any given point in time; when a square is encountered, the algorithm swaps the number with A[count] after increasing count ← count+1.

This algorithm is clearly $\Theta(n)$ with $\Theta(1)$ extra space, so it satisfies the constraints of (i) and (ii).

5i) Set A[1 ... n] to be an array containing n 1s.
index ← k // first person to die
for r ← 1 to j-1 do
    A[index] = 0
    skipped ← 0
    while skipped < k do
        index ← index+1
        if A[index] = 1 then
            skipped ← skipped + 1
    //index is the person slated to die next now
return index

5ii) Set A to be a CSLL with n nodes, each containing the corresponding prisoner's number.
for i ← 1 to k-1 do
    A.rotate()
//set head to first executed prisoner
for i ← 1 to j-1 do
    A.removeFirst()
    for i ← 1 to k-1 do
        A.rotate
return A.first()

6i) if n = 2 then
    return A[1] × A[2]
// if n > 2 there exist 2 non positive or 2 non negative numbers in A, so max product ≥ 0.
maxprod ← 0
for i ← 1 to n do
    for j ← i+1 to n do
        maxprod ← maxprod(A[i] × A[j], maxprod)
return maxprod

6ii) SORT (A)
// max product = product of 2 largest (nonnegative) numbers or
// 2 smallest (nonpositive) numbers (except possibly if n=2). so in general:
return max(A[1] × A[2], A[n-1] × A[n])

6iii) smallest ← A[1]
second_smallest ← A[1]
biggest ← A[1]
second_biggest ← A[1]

for i ← 1 to n do
    if A[i] ≤ smallest then
        second_smallest ← smallest
        smallest ← A[i]
    if A[i] ≥ biggest then
        second_biggest ← biggest
        biggest ← A[i]
return max(smallest × second_smallest, biggest × second_biggest))